Tuple, Dictionary and Set

ภาควิชาวิศวกรรมคอมพิวเตอร์
จุฬาลงกรณ์มหาวิทยาลัย
๒๕๕๙

Topics

- Tuple
- การเข้าถึงข้อมูลใน Tuple
- การทำงานกับ Tuple
- Dictionary
- การเพิ่ม ลด ข้อมูลใน dictionary
- การทำงานกับ dictionary
- Set
- การเพิ่ม ลด ข้อมูลใน set
- Set Operations

Tuple

Tuple

- Tuple: ลักษณะทั่วไป เหมือน List ยกเว้น
  - Tuple เปิดและปิดด้วยวงเล็บ
  - Tuple **สร้างแล้วปลี่ยนแปลงไม่ได้** (เหมือน String)

```
primes = [2,3,5,7,11,13,17]              #list
start_date = (10,"JAN",2015)             #tuple
student_score = (5837263721,38.5)        #tuple
price = (256.75,)                        #tuple
price = (256.75)  # ได้จำนวนเต็ม เหมือน price = 256.75
```

การเข้าถึงข้อมูลใน tuple

- เข้าถึงได้เหมือน list เช่น

```
>>> book = ("Data Science", 149190142X",28.79,10)
>>> book[0]
'Data Science'
>>> book[0:2]
('Data Science', '149190142X')
>>> book[-1]
10
>>> book[-2:-1]
(28.79,)
```

เปลี่ยนแปลงข้อมูลใน Tuple ไม่ได้

- สร้างแล้วเปลี่ยนแปลงไม่ได้ (เหมือนกับ string)

```
>>> s = "HELLo World"
>>> s[1:4] = "ell"
Traceback (most recent call last):
  File "<pyshell#95>", line 1, in <module>
    s[1:4]="ell"
TypeError: 'str' object does not support item
assignment

>>> book = ("Data Science", 149190142X",28.79,10)
>>> book[0] = "Info Science"
Traceback (most recent call last):
  File "<pyshell#97>", line 1, in <module>
    book[0] = "Info Science"
TypeError: 'tuple' object does not support item
assignment
```

บวก คือ การต่อ tuples แล้วสร้าง tuple ใหม่

```
>>> a = [1,2]   # นี่คือ list
>>> b = [3,4]
>>> c = a + b   # c เป็น list ใหม่ ได้จากการนำ a ต่อกับ b
>>> print(c)
[1, 2, 3, 4]

>>> title = ("Data Science",) # นี่คือ tuple
>>> isbn = ("149190142X",)
>>> price = (28.79,)
>>> amount = (10,)
>>> book1 = title+isbn+price+amount #สร้าง tuple ใหม่
>>> book1
('Data Science', '149190142X', 28.79, 10)
```

a,b,c คือ (a,b,c)



```
>>> (x,y) = (100,120) #การให้ค่ากับ tuple
>>> x,y = 100,120   # ระบบแปลง x,y เป็น (x,y)
                    # และแปลง 100,120 เป็น (100,120)
```

```
>>> student1 = ("Somsak","Rakrian",35)
>>> name,surname,age = student1
>>> name
'Somsak'
>>> surname
'Rakrian'
>>> age
35
```

```
>>> x,y = 100,120
>>> x,y = y,x
>>> x,y
(120,100)
```

สลับค่า

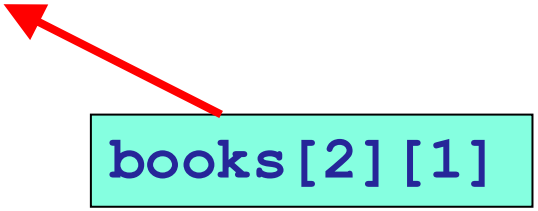
List of Tuples

- มักใช้ tuple เก็บกลุ่มข้อมูลที่สัมพันธ์กัน
  - เช่น หนังสือ ประกอบด้วย ชื่อ, ISBN, ราคา
- ใช้ list of tuples เก็บรายการของกลุ่มข้อมูล
  - เช่น รายการของหนังสือในร้าน

```
>>> books=[ \
        ("Data Science",    "149190142X", 28.79), \
        ("Learning Python","1449355730", 37.06), \
        ("Data Analysis",   "1449319793", 27.68)  ]

>>> for (title,isbn,price) in books:
      print( (isbn,title) )

('149190142X', 'Data Science')
('1449355730', 'Learning Python')
('1449319793', 'Data Analysis')
```



Dictionary (dict)

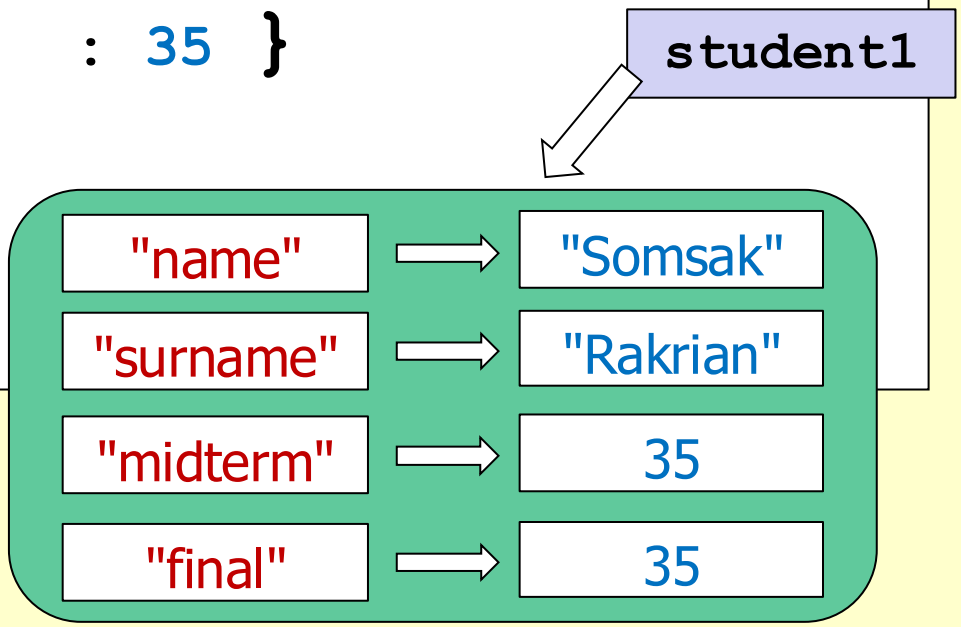
- list และ tuple
  - ใช้จำนวนเต็มเป็น index ในการเข้าถึงข้อมูลย่อยภายใน
  - student = (5730000021, "พรศักดิ์")
  - ต้องจำว่า student[0] คือรหัสนิสิต, student[1] คือชื่อ
- dict
  - index ไม่จำเป็นต้องเป็นจำนวนเต็ม
  - student = { "id" : 5730000021, "name" : "พรศักดิ์" }
  - print(student["id"]) ได้ 5730000021
    print(student["name"]) ได้ "พรเทพ"
    student["name"] = "พรนภา" เปลี่ยนชื่อได้เลย
  - อ่านเข้าใจกว่ามาก ๆ

Dictionary

Dictionary (dict)

- ที่เก็บข้อมูลของ คู่ลำดับ คีย์กับค่า (key-value pair)
- key ห้ามซ้ำ value ซ้ำได้

```
>>> student1 = { "name"    : "Somsak",   \
                 "surname" : "Rakrian",  \
                 "midterm" : 35,         \
                 "final"   : 35 }
>>> student1["name"]
'Somsak'
>>> student1["midterm"]
35
```



dict : การเพิ่ม key-value pair

```
>>> student1={}   # สร้าง dict ว่าง ๆ
>>> student1["name"]    = "Somsak"
>>> student1["surname"]= "Rakrian"
>>> student1["midterm"]= 35
>>> student1["final"]   = 40
>>> student1
{'name': 'Somsak', 'surname': 'Rakrian',
'midterm': 35, 'final': 40}
```

เพิ่ม key-value pairs

dict : บริการ keys(), values(), items()

```
>>> student1={"name":"Somsak","surname":"Rakrian"
,"midterm":35,"final":40}

>>> student1.keys()   # ได้ list ของ keys
dict_keys(['name', 'surname', 'midterm', 'final'])

>>> student1.values() # ได้ list ของ values
dict_values(['Somsak', 'Rakrian', 35, 40])

>>> student1.items() # ได้ list ของ tuples(key,value)
dict_items([('name', 'Somsak'), ('surname',
'Rakrian'), ('midterm', 35), ('final', 40)])
```

dict : ใช้ for ... in ....

```
>>> student1={"name":"Somsak","surname":"Rakrian"
,"midterm":35,"final":40}
```

```
>>> for key in student1 :
        print(key)

name
surname
midterm
final
```

```
>>> for key in student1.keys() :
        print(key)

name
surname
midterm
final
```

```
>>> for v in student1.values() :
        print(v)

Somsak
Rakrian
35
40
```

```
>>> for (k,v) in student1.items():
        print( k,v )

name Somsak
surname Rakrian
midterm 35
final 40
```

dict : บริการ update() ปรับปรุงข้อมูล

```
>>> en2th = {"attend":"เข้าร่วม", \
             "attendee":"ผู้เข้าร่วม", \
             "attendance":"การเข้าชั้นเรียน"}
>>> d = {"attendance":"การเข้างาน", \
         "attendant":"ผู้ต้อนรับ"}
>>> en2th.update(d)
>>> en2th
{'attend': 'เข้าร่วม', 'attendance': 'การเข้างาน',
'attendee': 'ผู้เข้าร่วม', 'attendant': 'ผู้ต้อนรับ'}
```

เก็บค่าใหม่ในกรณี key เหมือนกัน

key,value จาก dictionary ใหม่

dict : บริการ pop(key) ลบข้อมูล:

```
>>> en2th = {"attend":"เข้าร่วม", \
             "attendee":"ผู้เข้าร่วม", \
             "attendance":"การเข้าชั้นเรียน"}

>>> value = en2th.pop("attendee")
>>> value
'ผู้เข้าร่วม'

>>> en2th
{'attend': 'เข้าร่วม','attendance': 'การเข้าชั้นเรียน'}
```

dict : บริเวณค้น key ใน dict

```
>>> en2th = {"attend":"เข้าร่วม", \
             "attendee":"ผู้เข้าร่วม", \
             "attendance":"การเข้าชั้นเรียน"}
>>> "attend" in en2th
True

>>> "attending" in en2th
False

>>> "เข้าร่วม" in en2th
```

การใช้ dict

- แบบที่ 1 : ใช้ dict เก็บข้อมูลที่ประกอบด้วยข้อมูลย่อย
  - ให้ value เก็บข้อมูลย่อย
  - ให้ key เก็บความหมายของข้อมูลย่อย

```
student = {"id": 5730291821, \
           "name":"Somsak Rakrian"}
```

- แล้วใช้ list of dict เก็บรายการของกลุ่มข้อมูล

```
fin = open("students.txt")
students = []
for line in fin:
  (id,name) = line.split()
  students.append( {"id":id, "name":name} )
fin.close()
```

การใช้ dict

- แบบที่ 2 : ใช้ dict เสมือนเป็นฟังก์ชัน f(x)
  - x เป็น key ส่วน f(x) เป็น value
  - dict เก็บคู่ข้อมูลต่าง ๆ ที่มีความสัมพันธ์กัน

```
en2th = {"attend":"เข้าร่วม", \
         "attendee":"ผู้เข้าร่วม", \
         "attendance":"การเข้าชั้นเรียน"}
```

```
en2th = {"MO":"จ",  "TU":"อ", \
         "WE":"พ",  "TH":"พฤ", \
         "FR":"ศ", "SA":"ส", "SU":"อา" }
```

```
nicknames = {"Robert":"Bob", "Thomas":"Tom", \
         "Donald":"Don", "Elizabeth":"Betty",\
         "Jennifer":"Jen", "Richard":"Dick"}
```

ตัวอย่างการใช้ dict

เก็บคะแนนสอบย่อยต่าง ๆ ของนิสิตแต่ละคน ใช้ id เป็น key

```
points = {5730129292l: [10,9,10,8,2], \
          5730397772l: [9,10,8,2,10], \
          5731027822l: [9,10,10,8,10] }
```

เก็บรายละเอียดสินค่า ใช้รหัสสินค้า เป็น key

```
products = {
  "P10291": {"desc":"mem 8GB", "price":99}, \
  "P10872": {"desc":"mouse",   "price":129}, \
  "C83211": {"desc":"hdmi cable", "price":399} }
```

เก็บความสูงของพิกัดบนแผนที่ ใช้พิกัด (x,y) เป็น key

```
altitudes = { (10.8382,20.2882):12493.888, \
              (10.2376,19.3462):25352.683, \
              (11.0383,20.7272):11019.222 }
```

ใช้ tuple เป็น key ได้ ใช้ list เป็น key ไม่ได้

ตัวอย่าง : เปลี่ยนชื่อย่อวันในภาษาอังกฤษเป็นไทย

```
thaiD = [ "จ", "อ", "พ", "พฤ", "ศ", "ส", "อา" ]
engD  = [ "MO", "TU", "WE", "TH", "FR", "SA", "SU" ]
eng = input("Day = ")
if eng in engD :
  k = engD.index( eng )   # คืน index ของ engD ที่เก็บ eng
  print( thaiD[k] )
else :
  print( "Invalid day" )
```

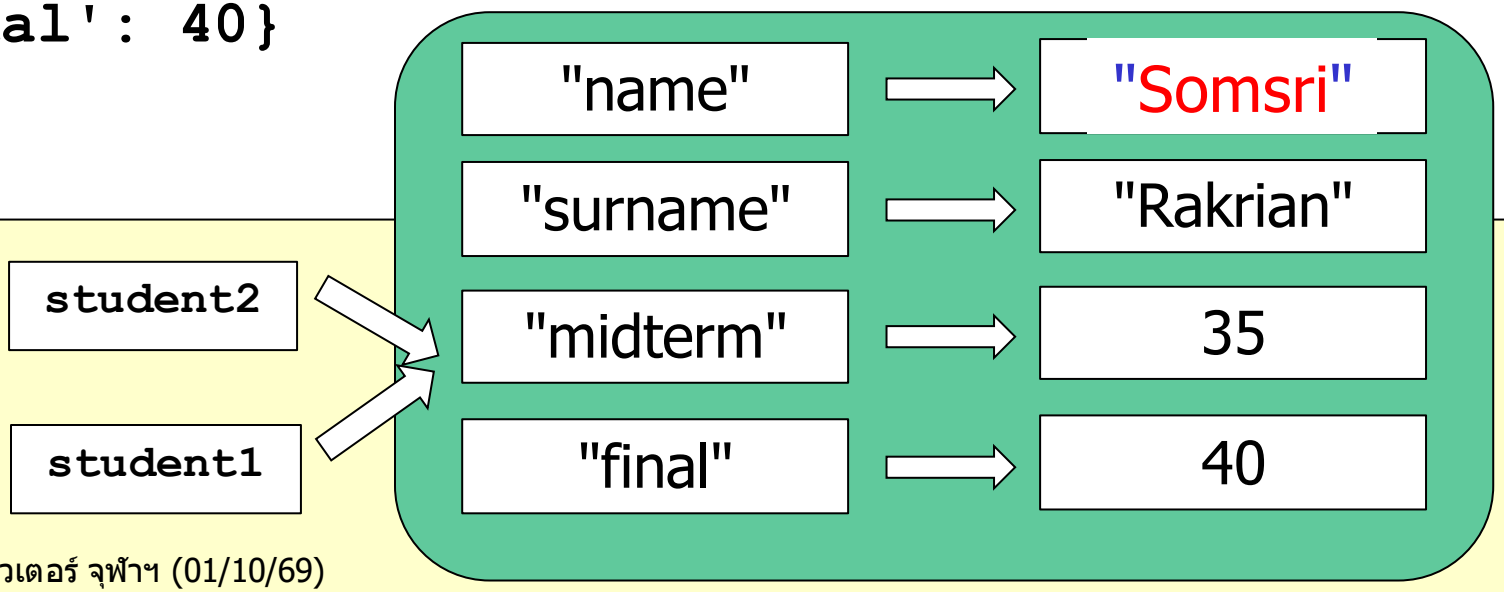
thaiD[จำนวนเต็ม]

```
en2th = {"MO":"จ",  "TU":"อ", "WE":"พ", \
         "TH":"พฤ", "FR":"ศ", "SA":"ส", "SU":"อา" }
eng = input("Day = ")
if eng in en2th:
  print( en2th[eng] )
else :
  print( "Invalid day" )
```

en2th[สตริง]

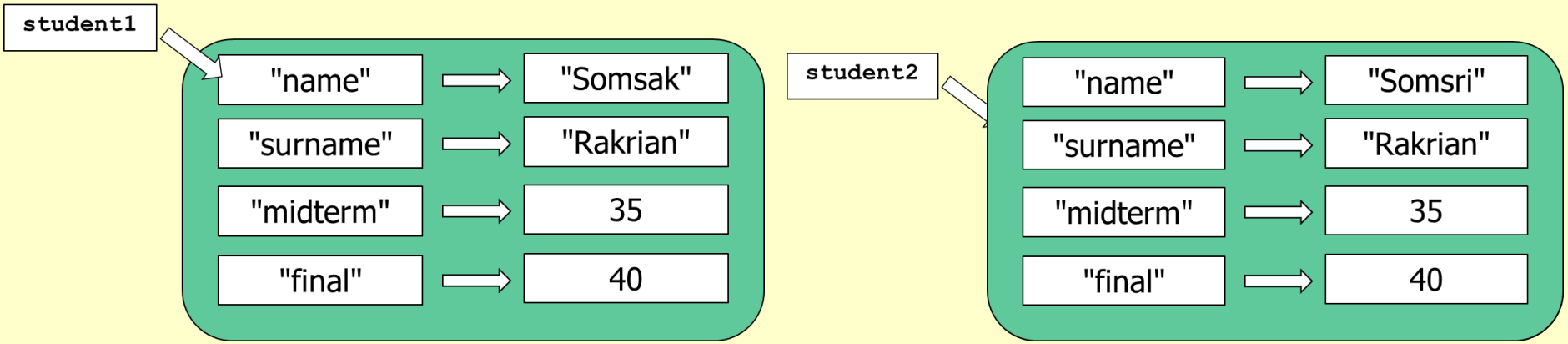
เพิ่มเติม : dict2 = dict1 ใช้ dict ร่วมกัน

```
>>> student1={"name":"Somsak","surname":"Rakrian"
,"midterm":35,"final":40}
>>> student2 = student1
>>> student2["name"] = "Somsri"
>>> student2
{'name': 'Somsri', 'surname': 'Rakrian', 'midterm':
35, 'final': 40}
>>> student1
{'name': 'Somsri', 'surname': 'Rakrian', 'midterm':
35, 'final': 40}
```



เพิ่มเติม : dict2 = dict(dict1) คือการ copy dict

```
>>> student1={"name":"Somsak","surname":"Rakrian"
,"midterm":35,"final":40}
>>> student2 = dict(student1)
>>> student2["name"] = "Somsri"
>>> student2
{'name': 'Somsri', 'surname': 'Rakrian',
'midterm': 35, 'final': 40}
>>> student1
{'name': 'Somsak', 'surname': 'Rakrian',
'midterm': 35, 'final': 40}
```



ตัวอย่าง : นับจำนวนตัวอักษรใน String

```
>>> s = "abracadabra"
>>> counts = {} #จะใช้ตัวอักษรเป็น key จำนวนที่นับได้เป็น value
>>> for char in s:
      if char in counts:
            counts[char] += 1
      else:
            counts[char] = 1

>>> print(count)
{'c': 1, 'a': 5, 'b': 2, 'd': 1, 'r': 2}
```

ตัวอย่างโจทย์

- ค้นหาชื่อ-นามสกุล จากเลขประจำตัว
  - ให้อ่านข้อมูลจากไฟล์ (หรือแป้นพิมพ์) ซึ่งเก็บข้อมูลเลขประจำตัวนิสิต 10 หลัก ตามด้วยชื่อ นามสกุล (อาจไม่ได้คั่นด้วยเว้นวรรคตัวเดียว เช่น กรณีมีชื่อกลาง หรือนามสกุลมีเว้นวรรค)
  - ให้ค้นหาชื่อ-นามสกุลจากเลขประจำตัว
  - **ใช้ list ไม่ดี ช้า ต้อง search**
  - **ใช้ tuple ไม่เหมาะ เพราะอ่านไป เพิ่มไปไม่ได้**
  - **ใช้ dict ที่เก็บเลขประจำตัวเป็น key สะดวก+เร็ว**
- ค้นหาเลขประจำตัว จากชื่อ-นามสกุล
  - อ่านข้อมูลเหมือนข้างบน
  - **ใช้ dict ที่เก็บชื่อ-นามสกุลเป็น key สะดวก+เร็ว**

ตัวอย่างโจทย์

- ค้นหาชื่อ-นามสกุล จากเลขประจำตัว หรือ ค้นหาเลขประจำตัวจากชื่อ-นามสกุล
  - ค้นจากเลขประจำตัวบ่อยกว่า
    - ใช้เลขประจำตัวเป็น key
    - ใช้ sequential search เมื่อต้องการค้นจากชื่อนามสกุล
  - ค้นจากชื่อนามสกุลบ่อยกว่า
    - ใช้ชื่อนามสกุลเป็น key
    - ใช้ sequential search เมื่อต้องการค้นจากเลขประจำตัว

Set

set

- คล้าย dict แต่เก็บเฉพาะ key ไม่มี value
- set ไม่เก็บข้อมูล ซ้ำ
- ทำ union, intersection, subtract ได้
- การค้นด้วย in ใน set เร็วกว่า list มาก

สร้าง Set ใหม่

```
>>> s0 = {"a","b","c"}          # สร้าง set ใหม่
>>> s = set( s0 )               # สร้างจาก set
>>> s = set(["a","b","c"]) # สร้างจาก list
>>> s
{'c', 'a', 'b'}
>>> s = set("abc")              # สร้างจาก string
>>> s
{'c', 'a', 'b'}
>>> s = set(["a","b","c","a","b"]) # กรณีมีข้อมูลซ้ำ
>>> s
```

set : บริการ add( ) เพิ่มข้อมูล

```
>>> s = {"a","b","c"}
>>> s.add("d")
>>> s
{'c', 'a', 'b', 'd'}
>>> s.add("d")        # เพิ่มข้อมูลที่มีอยู่แล้ว ไม่มีผลอะไร
>>> s
{'c', 'a', 'b', 'd'}
```

set : บริการ update() ปรับปรุงข้อมูล

```
>>> s = {"a","b","c"}
>>> s.update( ["d","e","f"] )   # update จากอีก list
>>> s
{'c', 'a', 'f', 'e', 'd', 'b'}
>>> s.update( {"e","f","g"} )   # update จากอีก set
>>> s
{'c', 'a', 'f', 'e', 'g', 'd', 'b'}
```

set : บริการ in, not in เพื่อค้นข้อมูลใน set

- x in myset x อยู่ใน myset หรือไม่
- x not in myset x ไม่อยู่ใน myset ใช่หรือไม่

```
blocked_numbers = {
      0891137635, 0892730928, 022180000, \
      0902918383, 0210398382, 088928199 }

while True :
   number = wait_for_calling()
   if number not in blocked_numbers :
      wait_for_receiving()
   else :
      record_block_call( number )
      disconnect_call()
```

set : บริการ remove, discard ลบข้อมูล

```
>>> s = set(["a","b","c"]) # สร้างจาก list
>>> s.remove("d") # เกิด Error ถ้าไม่มีข้อมูลอยู่
Traceback (most recent call last):
  File "<pyshell#152>", line 1, in <module>
    s.remove("d")
KeyError: 'd'
>>> s.discard("d") # ไม่เกิด Error ถ้าไม่มีข้อมูลอยู่
>>> s
{'c', 'a', 'b'}
```

```
>>> s.remove("a")
>>> s
{'c', 'b'}
>>> s.discard("b")
>>> s
{'c'}
```

ควรค้นให้มั่นใจก่อน
ว่ามีตัวที่จะถูกลบ
ก่อนลบ

set : บริการ union, intersection, difference

```
>>> a = {"a","b","c"}
>>> b = {"b","c","d"}
>>> a.union(b)          # a | b
{'c', 'a', 'd', 'b'}
>>> a.intersection(b) # a & b
{'c', 'b'}
>>> a.difference(b)    # a - b
{'a'}
>>> b.difference(a)    # b - a
{'d'}
>>> a-b
{'a'}
>>> b-a
{'d'}
>>> a ^ b              # (a | b)-(a & b)
{'d', 'a'}
```

เพิ่มเติม : บริการ issubset, issuperset

```
>>> a = set("communicatively")
>>> b = set("aeiou")
>>> b.issubset(a)
True
>>> a.issuperset(a)
True
>>> a.issuperset(b)
True
>>> a.issubset(b)
False
>>> b.issuperset(a)
False
>>> "e" in a
True
>>> "x" not in a
True
```

เพิ่มเติม : set2 = set1 กับ set2 = set(set1)

```
>>> s1 = {"a","b","c"}
>>> s2 = set(s1)                    # s2 สร้างจาก s1
>>> s2.add("d")                     # เพิ่ม "d" ใน s2
>>> s1
{'c', 'a', 'b'}
>>> s2
{'c', 'a', 'b', 'd'}
```

```
>>> s1 = {"a","b","c"}
>>> s2 = s1                         # s2 เป็นตัวเดียวกับ s1
>>> s2.add("d")                     # เพิ่ม "d" ใน s2
>>> s2
{'c', 'a', 'b', 'd'}
>>> s1
```

ตัวอย่าง: Pair Sum

- กำหนดข้อมูลชุดหนึ่งเป็นตัวเลขจำนวนเต็ม (ไม่ซ้ำกัน) และค่า K
  - ให้หาข้อมูลสองตัวในชุดที่บวกกันได้ K
  - ถ้าหาไม่พบ ให้พิมพ์ “Not Found”
- ถ้าใช้ loop สองชั้นตามด้านล่างกับปริมาณข้อมูลไม่มาก ก็ทำงานได้ แต่ถ้าปริมาณข้อมูลมาก จะช้ามาก จะแก้ไขโปรแกรมอย่างไร?

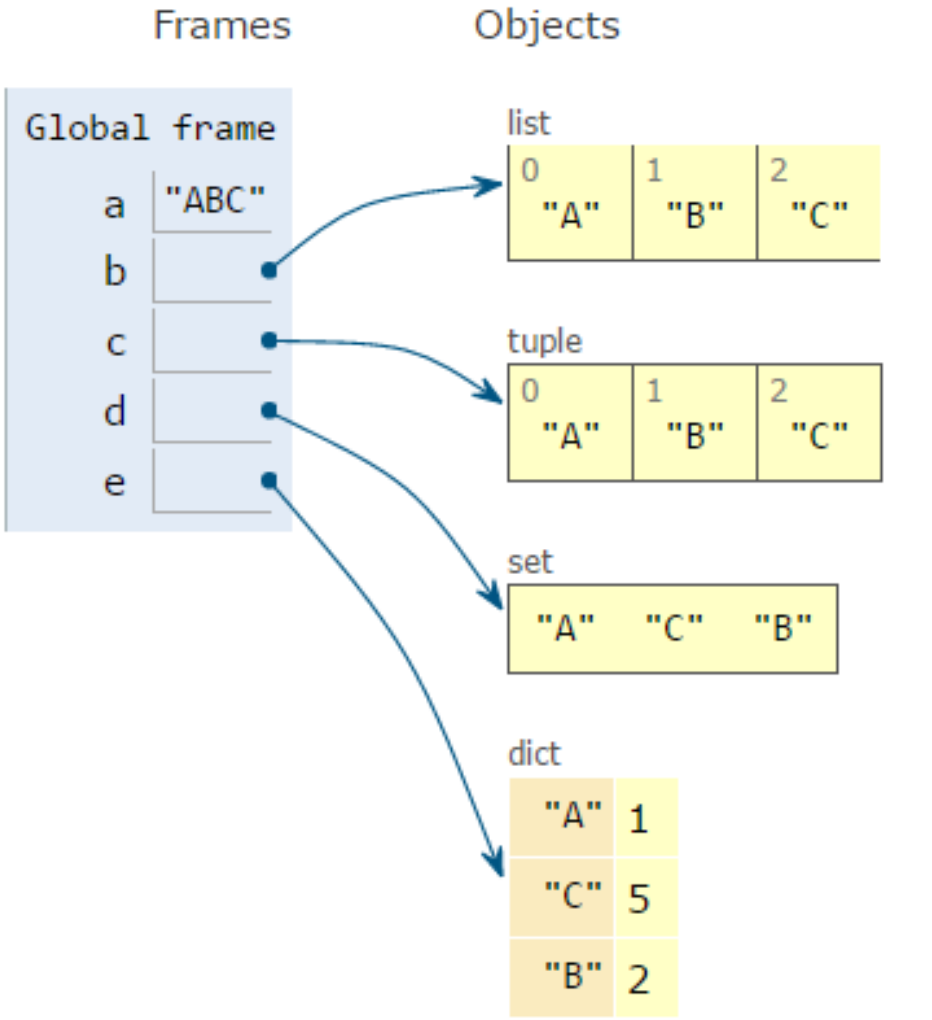
```
d=[159, 557, 837, 42, 846, 490, 567, 32, 4, 139]
result = 599
found = False
for i in range(len(d)):
    for j in range(i+1,len(d)):
        if d[i]+d[j] == result:
            found = True
            print(d[i],d[j])
if not found:
    print("Not Found")
```

หาจำนวนนักเรียนที่เรียนสองวิชา

- ให้อ่านข้อมูลจากแป้นพิมพ์ โดยอ่านข้อมูลเป็นบรรทัด แต่ละบรรทัดมีรหัสนักเรียนคั่นด้วยเว้นวรรค ตามด้วยรหัสวิชา
- เมื่อพบว่า รหัสนักเรียนเป็น -1 ให้หยุดอ่าน
- จากนั้นให้อ่านรหัสวิชาสองรหัสวิชา แล้วให้แสดง
  - จำนวนนักเรียนที่เรียนทั้งสองวิชานั้น
  - จำนวนนักเรียนที่เรียนวิชาเดียว
  - จำนวนนักเรียนทั้งหมด ซึ่งเรียนวิชาใดวิชาหนึ่งหรือทั้งสองวิชา

สรุป string, list, tuple, set, dict

```
1  a="ABC"
2  b=["A","B","C"]
3  c=("A","B","C")
4  d={"A","B","C"}
5  e={"A":1,"B":2,"C":5}
```



สรุปการใช้งาน list, tuple, dict, set

	list	tuple	dict	set
การใช้	- ลำดับของข้อมูลมีความหมาย อาจมีการเปลี่ยนแปลง - ข้อมูลในรายการมักมีความหมายเดียวกัน	- ลำดับของข้อมูลมีความหมาย - สร้างแล้วไม่เปลี่ยนแปลง - ข้อมูลใน tuple มักมีความหมายต่างกัน	- เก็บข้อมูลเป็นคู่ๆ key-value โดยใช้ key เข้าถึงข้อมูลเพื่อให้ได้ value มาใช้งาน	- เก็บข้อมูลไม่ซ้ำ ลำดับของข้อมูลไม่มีความหมาย เพื่อตรวจสอบว่า มีข้อมูลหรือไม่ รองรับ set operations
การเข้าใช้ข้อมูล	ใช้จำนวนเต็มระบุตำแหน่ง `d[i]`	ใช้จำนวนเต็มระบุ `d[i]`	ใช้ key เป็นตัวระบุตำแหน่งข้อมูล `d[key]`	ต้อง `for...in...` เพื่อแจงข้อมูล
การค้นด้วย `in`	ค้นจากซ้ายไปขวาช้า	ค้นจากซ้ายไปขวาช้า	มีวิธีค้นที่เร็วมาก	มีวิธีค้นที่เร็วมาก
การสร้าง	`x =[1,2,3,4]`	`t = (1,2,3,4)`	`d = { "k1":1,` `"k2":2 }`	`s = {1,2,3,4}`

การเพิ่มข้อมูล	`x.append(3)`
	`x.insert(1,99)`
		สร้างแล้วเปลี่ยนแปลงไม่ได้ ต้องสร้างใหม่
		`t = t + (4, )`
			`d["k1"] = 1`
			`d["k2"] = 2`
			หรือใช้ `update`
				`s.add(3)`

เลือกใช้อะไรดี ?

- ต้องการเก็บรายชื่อนิสิตคณะวิศวฯทุก ๆ รุ่น ต้องการให้บริการหาว่าชื่อที่ระบุอยู่รุ่นใด
  - ใช้ dict ที่มี key เป็นชื่อ value เป็นรุ่น
- ในการสมัครสอบ admission ผู้สมัครต้องเลือกสาขาที่ต้องการ เลือกได้ 4 ลำดับมากสุด หลังการกรอกข้อมูลแล้ว ระบบจะกำหนดหมายเลขการสมัคร appID ให้กับผู้สมัครแต่ละคน เราควรเก็บข้อมูลนี้อย่างไรต้องการให้บริการหาว่าลำดับที่ k ของหมายเลข appID คืออะไร

เลือกใช้อะไรดี ?

- จากข้อมูลสลากกินแบ่งในอดีตอันมากมาย ต้องการให้บริการหาว่าหมายเลขรางวัลที่ 1 อะไรบ้าง ที่ลงท้ายด้วยเลขสองหลักที่กำหนดให้

- มีข้อมูลคะแนนสอบของวิชา 2110101 ย้อนหลัง 30 ปี (เช่น 5331010121 ได้ 98.2, 4730002021 ได้ 50.33, ...) ต้องการให้บริการสอบถามว่า มีนักเรียนกี่คนได้คะแนนสอบเท่ากับ x (ไม่สนใจว่าใครได้คะแนนเท่าไร)